ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Type Classes — Class Challenge Set Feedback

Challenge 1

```haskell
data Color  = Red | Green | Blue
data Tree a = Leaf | Node (Tree a) a (Tree a)

instance Show Color where
  show Red   = "Red"
  show Green = "Green"
  show Blue  = "Blue"

instance Show a => Show (Tree a) where
  show Leaf         = "Leaf"
  show (Node l x r) = "Node (" ++ show l ++ ") " ++ show x ++ " (" ++ show r ++ ")"
```

For `Tree` the instance is *qualified* over `Show a`: we cannot print a tree of `a`s unless we know how to print an `a`. The qualified context becomes a dictionary parameter under the dictionary-passing translation.

`deriving Eq` would synthesise:

- For `Color`: a structural equality that returns `True` exactly on `(Red, Red)`, `(Green, Green)`, `(Blue, Blue)` and `False` on every other pair.
- For `Tree`: `Leaf == Leaf = True`; `Node l1 x1 r1 == Node l2 x2 r2 = l1 == l2 && x1 == x2 && r1 == r2`; mixed-constructor cases are `False`. The compiler attaches the natural qualifier `Eq a => Eq (Tree a)` because comparing the `a` payload requires `Eq a`.

In both cases the derived instance is *purely structural*: equality of constructors first, then componentwise equality of fields.

Challenge 2

```haskell
class Container c where
  empty  :: c a
  insert :: Eq a => a -> c a -> c a
  member :: Eq a => a -> c a -> Bool
```

Why the `Eq a` qualifier? `member` has to *test* an element for equality against the contents of the container, so it intrinsically depends on `Eq a`. `insert` is more subtle: for ordinary lists with duplicates it does not need `Eq a`, but for `Set` it does (we must check whether the element is already present). Since instance methods may not introduce per-instance constraints beyond those declared in the class, we hoist `Eq a` into the class signature of `insert` as well; the list instance just ignores it.

`empty` is a polymorphic value of type `c a`: it carries no element, so it carries no constraint.

Challenge 3

```haskell
instance Container [] where
  empty       = []
  insert x xs = x : xs
  member x xs = x `elem` xs

newtype Set a = Set [a]

instance Container Set where
  empty                             = Set []
  insert x (Set xs) | x `elem` xs   = Set xs
                    | otherwise     = Set (x : xs)
  member x (Set xs)                 = x `elem` xs
```

The two instances make the trade-off explicit: the list instance ignores `Eq a` for `insert`; the `Set` instance uses it to enforce no-duplicates.

Challenge 4

```haskell
data Json
  = JString String | JNumber Double | JBool Bool
  | JArray  [Json] | JObject [(String, Json)]
```

(a) Expressing `Json` values.

```haskell
example :: Json
example = JObject
  [ ("title",       JString "Advanced Programming Languages")
  , ("year",        JNumber 2026)
  , ("instructors", JArray [JString "M. Bugliesi"])
  ]
```

(b) Conversion functions.

```haskell
int2json     :: Int -> Json
int2json n   =  JNumber (fromIntegral n)

string2json  :: String -> Json
string2json  =  JString

bool2json    :: Bool -> Json
bool2json    =  JBool

ints2json    :: [Int] -> Json
ints2json xs =  JArray (map int2json xs)

strings2json :: [String] -> Json
strings2json xs = JArray (map string2json xs)
```

(c) The `Movie` datatype.

```haskell
data Movie = Movie
  { title     :: String
  , year      :: Int
  , directors :: [String]
  }

movie2json :: Movie -> Json
movie2json m = JObject
  [ ("title",     string2json  (title m))
  , ("year",      int2json     (year m))
  , ("directors", strings2json (directors m))
  ]

-- > movie2json (Movie "Inception" 2010 ["Christopher Nolan"])
-- JObject [("title",JString "Inception"),
--          ("year",JNumber 2010.0),
--          ("directors",JArray [JString "Christopher Nolan"])]
```

(d) Type class `Jencode`. The shared shape of every conversion above is "give me a value, I give you back a `Json`":

```haskell
{-# LANGUAGE FlexibleInstances #-}

class Jencode a where
  encode :: a -> Json

instance Jencode Int where
  encode n = JNumber (fromIntegral n)

instance Jencode Bool where
  encode = JBool

instance Jencode Char where           -- needed only to make the list instance work on String
  encode c = JString [c]              -- (effectively unused, see overlap below)

instance {-# OVERLAPPING #-} Jencode String where
  encode = JString

instance {-# OVERLAPPABLE #-} Jencode a => Jencode [a] where
  encode xs = JArray (map encode xs)

instance Jencode Movie where
  encode m = JObject
    [ ("title",     encode (title m))
    , ("year",      encode (year m))
    , ("directors", encode (directors m))
    ]
```

Two design points worth noting:

1. **The list instance is a single qualified instance**, `Jencode a => Jencode [a]` : one declaration covering all element types, with the qualifier expressing the genuine dependency; to encode a list of `a` 's, we need to know how to encode an `a` .
2. **`String = [Char]`** clashes with the generic list instance. This can be resolved using the *overlapping* directives to the compiler: an `OVERLAPPING` instance for `String` sends a string to `JString` directly, while the `OVERLAPPABLE` generic list instance applies to every *other* list (including `[String]` , which becomes `JArray` of `JString` s). This is the price of representing strings as character lists in Haskell — in a language that distinguished `String` from `[Char]` natively, the conflict would not arise.

(e) Dictionary-passing translation.

Each class member becomes a record field; each instance becomes a value of the corresponding record type; each qualifier becomes an extra parameter receiving the dictionary.

```
-- The dictionary type
data JencodeDict a = JencodeDict { encode_d :: a -> Json }

-- Instance values (ground instances)
intDict :: JencodeDict Int
intDict = JencodeDict { encode_d = \n -> JNumber (fromIntegral n) }

boolDict :: JencodeDict Bool
boolDict = JencodeDict { encode_d = JBool }

stringDict :: JencodeDict String
stringDict = JencodeDict { encode_d = JString }

-- Parametric instance: takes the dictionary for the element type,
-- returns the dictionary for the list type
listDict :: JencodeDict a -> JencodeDict [a]
listDict d = JencodeDict { encode_d = \xs -> JArray (map (encode_d d) xs) }

-- Movie dictionary: closes over the dictionaries of all field types
movieDict :: JencodeDict Movie
movieDict = JencodeDict
  { encode_d = \m -> JObject
      [ ("title",     encode_d stringDict            (title m))
      , ("year",      encode_d intDict               (year m))
      , ("directors", encode_d (listDict stringDict)  (directors m))
      ]
  }
```

The `listDict` value is the heart of the translation: a parametric instance becomes a *function from dictionaries to dictionaries*. The qualifier `Jencode a => Jencode [a]` in the source translates one-to-one into the arrow `JencodeDict a -> JencodeDict [a]`. The compiler decides, at every call site, which dictionary value to feed in: for `[String]` it picks `listDict stringDict`; for `[[Int]]` it would synthesise `listDict (listDict intDict)`, and so on.

A polymorphic user function translates the same way. Take

```
encodePair :: (Jencode a, Jencode b) => a -> b -> Json
encodePair x y = JArray [encode x, encode y]
```

After translation each constraint becomes an explicit dictionary parameter:

```
encodePair' :: JencodeDict a -> JencodeDict b -> a -> b -> Json
encodePair' da db x y = JArray [ encode_d da x, encode_d db y ]
```

Calling `encodePair' intDict stringDict 7 "hello"` is exactly what the source-level call `encode 7` and `encode "hello"` reduces to: the compiler has inserted the dictionaries on our behalf, and the dispatch is just record-field selection at runtime.